IRWA Final Project - Part 1 Report

Carla Royo Gómez (u233460) Ingrid Sigró Alonso(u232682) Rita Alonso Casablancas (u233565)

GitHub repository: https://github.com/Moca-15/irwa-26.git

Delivery TAG: IRWA-2025-part-1-2026]

Part 1: Text Processing and Exploratory Data Analysis

1. Introduction

The goal of Part 1 is to prepare an e-commerce fashion catalogue so that a search engine can index it and rank its results. The work has two blocks:

- Data preparation: pre-processing the free-text fields, deciding how to handle the categorical and numeric fields, and fixing the set of fields that every result must return.
- Exploratory data analysis (EDA): describing the dataset (missing values, text lengths, vocabulary, categories, brands, prices, ratings, stock) and analysing the validation labels that will be used for evaluation in Part 2.

All the code is in irwa-part1.ipynb. It was run on the full dataset of 28.080 products (fashion_products_dataset.json). Therefore, the numbers shown in our report are the outputs of that run.

2. Dataset

The dataset fashion_products_dataset.json contains product records from a fashion e-commerce catalogue (product URLs point to Flipkart). Each record has 17 fields: _id, actual_price, average_rating, brand, category, crawled_at, description, discount, images, out_of_stock, pid, product_details, seller, selling_price, sub_category, title, url.

Every pid is unique. Prices and discounts are stored as strings with irregular formats (for example "2.999" and "69% off"), product_details is a list of single-key dictionaries, and out_of_stock is a boolean. The field images is not needed for search and it is ignored.

The validation file validation_labels.csv contains 40 (query, product) pairs with binary relevance labels for two queries: query 1 is "women full sleeve sweatshirt cotton" and query 2 is "men slim jeans blue".

3. Part 1 - Data preparation

3.1 Text pre-processing (title, description, product details)

The three free-text fields go through the same four steps, in this order:

Step	Implementation	Reason
Lowercasing	str.lower()	Words such as Shirt, shirt and SHIRT must map to the same term.
Tokenization and punctuation removal	re.findall(r"[a-z0-9]+", text)	Keeps only runs of letters and digits. Punctuation disappears as a side effect, so no separate punctuation step is needed.
Stop-word removal	NLTK English stop-word list	Function words such as the, and, of, with occur in almost every product and carry no discriminative information.
Stemming	NLTK Porter stemmer	Groups inflected or derived forms (shirt/shirts) so that a query term matches all of them.

The same function, preprocess_text, must be applied to the user queries in Parts 2-4. Otherwise, a query term and the stored term would be different strings even when they mean the same thing.

Worked example, using the first product title and a test string:

Input: Women's Full-Sleeve Printed Sweatshirts (Pack of 2) - 100% cotton!
tokenize: ['women', 's', 'full', 'sleeve', 'printed', 'sweatshirts', 'pack', 'of', '2', '100', 'cotton']
remove stop: ['women', 'full', 'sleeve', 'printed', 'sweatshirts', 'pack', '2', '100', 'cotton']
stemmed: ['women', 'full', 'sleev', 'print', 'sweatshirt', 'pack', '2', '100', 'cotton']

From this example, we can extract some observations:

- The possessive "s" survives tokenization as a separate token, and is then removed as a stop word. This is not a bug, it is the intended behaviour.
- Porter stemming is aggressive. sleeve becomes sleev and printed becomes print. The stems are not real words, but they are consistent, so matching still works. Users never see stems, because results show the original fields, so this only affects the index and the query matching.
- Numbers are kept as tokens (2, 100). They may be useful for matching product specifications such as "pack of 2", so they were not removed.

3.2 Domain stop words

We decided that no domain-specific stop words were added. The set DOMAIN_STOPWORDS is defined but keeps empty.

Justification: words such as women, men, solid, multicolor appear in a very large share of the catalogue, which makes them look like candidates for removal. However, these are the words that separate the evaluation queries: query 2 is "men slim jeans blue", and if men were removed the query would reduce to "slim jeans blue" and would match women's jeans as well. The EDA shows that these terms are among the most frequent in the processed corpus: men occurs 16.835 times and women 17.570 times, and both are kept in the index.

Stop-word removal and the domain decision are therefore kept deliberately minimal. Adding words later only requires editing one set.

3.3 Output fields

Every search result must return the following fields, when they are present in the record: pid, title, description, brand, category, sub_category, product_details, seller, out_of_stock, selling_price, discount, actual_price, average_rating, url. That is 14 fields. They are defined once in OUTPUT_FIELDS, so the indexing, the ranking, and web-interface steps of later parts return the same information.

pid is kept unchanged in every structure, because the evaluation matches results to labels by pid.

3.4 Categorical fields: brand, category, sub_category, seller

Three ways of handling these fields were considered:

Option	**Advantages**	**Disadvantages**
Merge all fields into one text field	Simple; one index; no decision about weights.	Loses the origin of each term. A brand name that looks like an ordinary word (e.g. "Puma" vs. "puma") would be confused with product text. Cannot weight brand matches differently from description matches.
Index each field separately	Allows field-specific weights (e.g. title weighted above description); allows	More complex; the indices must be combined at ranking time; sparse fields (brand is missing for 7.2% of products)

Option	Advantages	Disadvantages
	exact filters on sub_category.	make the separate indices uneven.
Hybrid: free text for title/description/details, exact labels for categorical fields	Exact filtering by category, sub-category, brand or seller; avoids stemming brand names; keeps the text index small.	An exact label only matches if the user types it exactly; a query for "jeans" does not match the sub-category "Bottomwear" unless the two are linked explicitly.

Chosen option: hybrid approach. Each categorical field is normalised to a single lowercase label (whitespace collapsed, empty values set to None) and is not stemmed. The function normalise_label implements this. For example, "Clothing and Accessories" becomes clothing and accessories as one unit.

The EDA supports this choice for the following reasons:

- category carries almost no information. 27.118 of 28.080 products (96,6%) are "Clothing and Accessories". As a text term it would appear in almost every document and would add nothing to ranking.
- sub_category is informative and well-populated. It has no missing values and separates the catalogue into meaningful groups (Topwear 15.310; Bottomwear 3.666; Winter Wear 2.493). It is well suited to exact filtering.
- brand is unreliable as a text term. 2.009 products (7,2%) have no brand, and many brand strings look truncated at about 10 characters ("Purple Sta", "Solid Styl", "Black Beat", "Marca Disa"). Some are very short ("Pu", 798 products; "Amp", 585). Stemming such strings would make them harder to match, so they are kept as exact labels.
- seller is missing for 5,9% of products. It is kept as an exact label for completeness, but it is not expected to be important for relevance.

3.5 The product_details field

product_details is a list of single-key dictionaries, for example [{"Color": "Blue"}, {"Fabric": "Cotton Blend"}]. The function product_details_to_text converts it to text while keeping the keys, so that the value Cotton Blend is linked to the key Fabric:

Style Code 1005COMBO2 Closure Elastic Pockets Side Pockets Fabric Cotton Blend Pattern Solid Color Multicolor
-> ['style', 'code', '1005combo2', 'closur', 'elast', 'pocket', 'side', 'pocket', 'fabric', 'cotton', 'blend', 'pattern', 'solid', 'color', 'multicolor']

The text is then processed exactly like title and description. This field is the main source of attributes like colour, fabric and pattern, so it contributes substantially to the text vocabulary.

3.6 Numeric fields: selling_price, actual_price, discount, average_rating, out_of_stock

The numeric fields are parsed into real values by four functions:

- parse_price: removes thousands separators ("2.999" becomes 2999,0). Empty or malformed values become None.
- parse_discount: extracts the number before the percent sign ("69% off" becomes 69,0).
- parse_rating: converts the text to a float ("3,9" becomes 3,9).
- parse_bool: accepts real booleans and the strings "true"/"false".

We decided that the numeric fields are not indexed as text terms. A number such as 921 is not a word a user would type when searching. Indexing it would add many meaningless tokens and mix non-text values into the text index. Instead the values are stored as metadata and will be used in Part 3 for two purposes: filtering (for example, only products in stock, or price below a threshold) and ranking (for example, by rating or discount). If the team later wants to index them, the sensible approach is to bin them into labelled ranges, such as price_500_1000, and to justify the bin width in the report.

Missing values are kept as None and are excluded from the statistics that use them. Missing ratings (8,1%) are the most important case: a product without a rating should not be ranked as if it had a rating of zero.

3.7 Processing the whole corpus

The function process_document applies all of the above to one product and returns a dictionary with four parts: fields (processed tokens of the three text fields), labels (normalised categorical labels), numeric (parsed numeric values) and display (the original 14 output fields). Applied to the whole corpus it produced 28.080 documents, with 28.080 unique pid values, and every output field was present in the first document.

4. Part 2 - Exploratory Data Analysis

4.1 Missing values

Field	Empty values	Percentage
description	11.150	39,7%
average_rating	2.261	8,1%
brand	2.009	7,2%

Field	Empty values	Percentage
seller	1.643	5,9%
discount	855	3,0%
actual_price	777	2,8%
selling_price	2	0,0%
title, category, sub_category, product_details, out_of_stock, url, pid	0	0,0%

Interpretation. The most important gap is the description: almost two in five products have none. Search results for these products can only match on title and product details, so the title field must carry more weight in ranking. The lack of descriptions also explains why many description-based statistics below are dominated by zeros.

4.2 Length of titles and descriptions

Field (tokens, before stop-word removal)	Mean	Std	Min	25%	Median	75%	Max
Title	6,93	2,35	1	6	7	8	36
Description	30,71	45,84	0	0	11	50	693

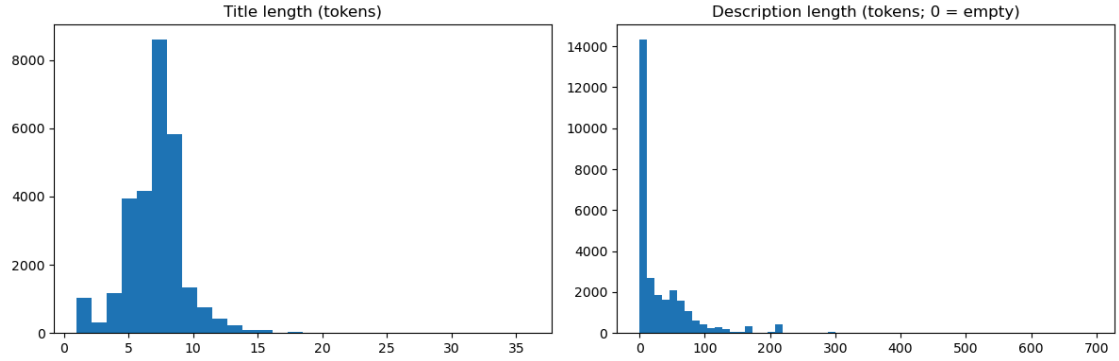


Figure 1. Distribution of title length (left) and description length (right) in tokens. Zero-length descriptions are the missing values.

Titles are short and consistent: most have 6 to 8 tokens. Descriptions are very uneven. The median is 11 tokens, but the mean is 30,7 and the longest description has 693 tokens, so the distribution is strongly right-skewed. A length normalisation will be needed in the ranking function, otherwise long descriptions will dominate scores.

4.3 Sentences and average sentence length

Sentences are counted with NLTK's Punkt tokenizer (sent_tokenize), which uses learned abbreviation and boundary rules rather than only punctuation. Results:

- Sentences per description: mean 1,89, median 1, maximum 31. Descriptions with no sentences are the empty ones (39,7% of the catalogue, see Section 4.1).
- Average sentence length: 16,23 tokens.

Descriptions are short, typically one or two sentences of about 16 tokens. Each sentence is a plausible unit for snippets in the results page.

4.4 Vocabulary size

The comparison uses the same fields (title and description) before and after processing:

Vocabulary	**Distinct terms**
Raw (lowercased tokens)	6.503
Processed (stop words removed, stemmed)	4.902

Pre-processing reduces the vocabulary by 1.601 terms (24,6%). The run did not separate the two steps, so we cannot say how much of the reduction comes from stop-word removal and how much from stemming. Stemming merges variants such as shirt/shirts, which makes matching less sensitive to the form of the word used in a query.

4.5 Most frequent terms

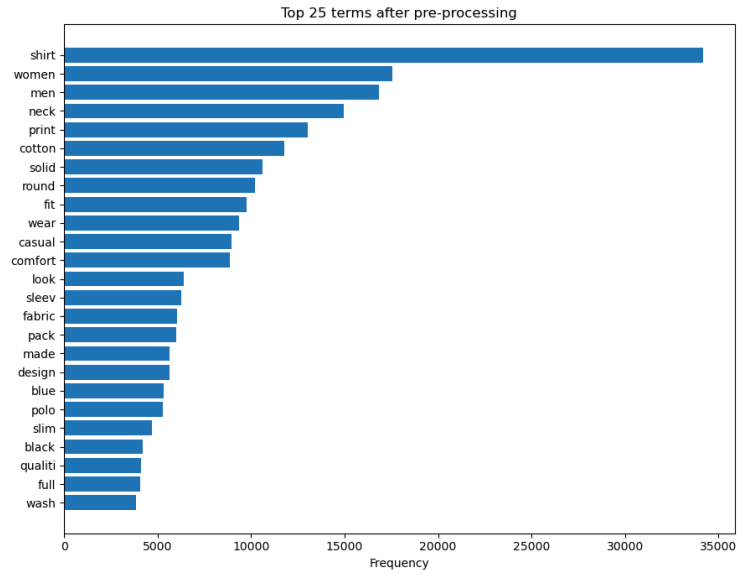
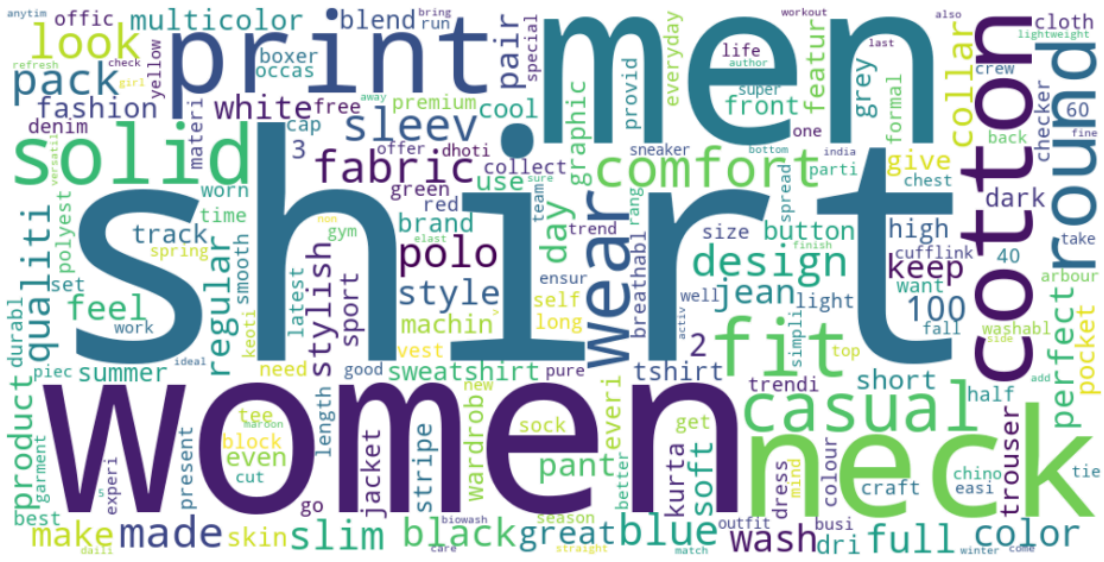


Figure 2. The 25 most frequent processed terms in title and description.

The most frequent term is shirt (about 34.000 occurrences), followed by women (17.570), men (16.835), neck, print, cotton, solid, round, fit, wear, casual and comfort. Among the frequent words are colour and fabric terms (blue, black, cotton) and product-type terms (polo, slim). The chart supports the decision in Section 3.2: the gender terms are among the most frequent, so removing them would drop two of the words that matter most for the evaluation queries.

Generic apparel terms (shirt, neck, fit, wear, casual) appear in many products and therefore discriminate little. Ranking with TF-IDF, which down-weights frequent terms, is expected to reduce their influence; this is one reason for using a weighted scheme in Part 3.

4.6 Word cloud


shirt
women
men
print
solid
wear
fit
casual
neck
cotton
round
comfort
fabric
design
polo
style
sleev
white
blue
black
slim
made
make
look
pack
fashion
feel
regular
stylish
sport
sweatshirt
tshirt
jean
button
high
keep
100
pant
jacket
stripe
wardrob
kurta
full
color
wash
dri
short
trendi
pocket
perfect
multicolor
blend
boxer
occas
pair
special
graphic
provid
everyday
collar
featur
front
grey
formal
cool
premium
free
cap
fabric
collect
use
dhoti
offer
sneaker
give
chest
dark
checker
cloth
lightweight
crew
back
60
denim
materi
worn
time
track
spring
green
red
brand
polyest
qualiti
durabl
set
work
summer
piec
product
present
length
tee
block
even
cut
go
best
make
made
skin
slim
black
great
blue
wash
dri
full
color
soft
craft
chino
easi
trouser
tie
colour
dress
outfit
busi
season
biowash
care
straight
match
winter
come
half
short
trendi
top
fall
washabl
arbour
take
cufflink
40
want
light
well
jean
design
button
high
keep
100
size
day
trend
ensur
self
long
machin
vest
need
good
pure
new
sock
get
everi
better
pant
wardrob
stripe
jacket
length
latest
smooth
keoti
gym
non
dhoti
offer
sneaker
give
chest
dark
checker
cloth
lightweight
crew
back
60
fine
india
bottom
parti
spread
one
super
life
author
last
workout
also
anytim
offic
bring
run
refresh
check
girl
yellow
away
cap
3

sub_category (top 10)	Products
Topwear	15.310
Bottomwear	3.666
Winter Wear	2.493
Innerwear and Swimwear	1.762
Clothing Accessories	1.742
Kurtas, Ethnic Sets and Bottoms	1.180
Men's Footwear	920
Fabrics	534
Sleepwear	122
Blazers, Waistcoats and Suits	103

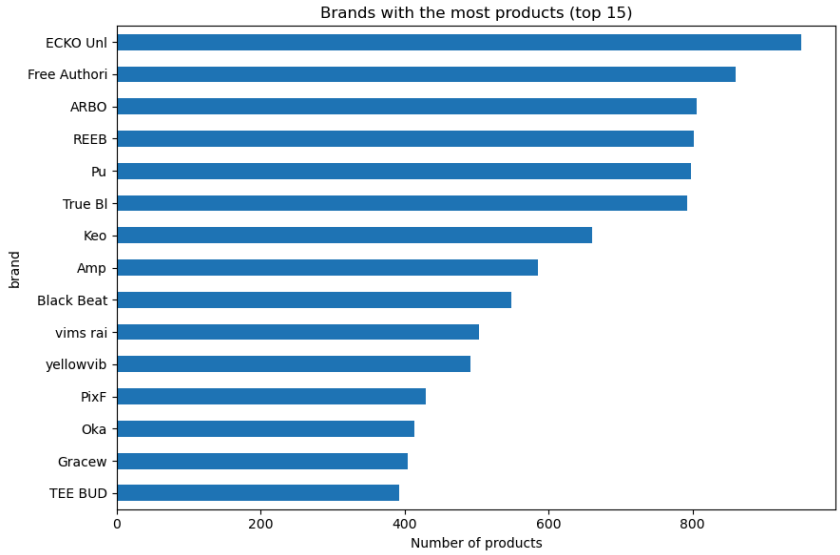


Figure 4. The 15 brands with the most products.

The category field has almost no variation, which confirms the decision in Section 3.4 to exclude it from text indexing. The sub-category is much more informative: Topwear alone is about 55% of the catalogue, so it will dominate any unfiltered result list.

The brand chart shows an important data-quality problem. Several of the most frequent "brands" (Pu, Keo, Amp, Oka, Gracew) are very short, and others (Free Authori, True Bl, vims rai) look cut off. The values appear to have been truncated during scraping. Brand matching will therefore be less reliable than the catalogue suggests, and this should be stated as a limitation of the results.

4.8 Ratings, prices, discounts and stock

	selling_price	actual_price	discount (%)	average_rating
Count	28.078	27.303	27.225	25.819
Mean	705,64	1.455,53	50,26	3,63
Std	549,68	939,98	16,89	0,66
Min	99	150	1	1,00
25%	390	849	40	3,20
Median	545	1.199	53	3,80
75%	820	1.799	63	4,10
Max	7.999	12.999	87	5,00

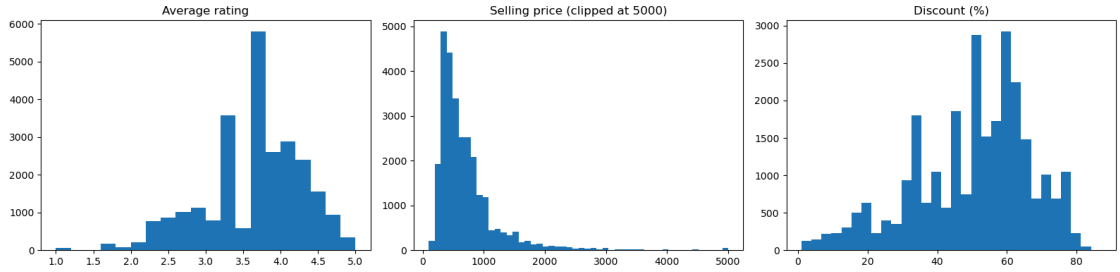


Figure 5. Distributions of average rating, selling price and discount.

Out of stock. 5,85% of products (1.644 of 28.080) are out of stock. The share varies by sub-category:

sub_category (at least 50 products)	Share out of stock	Products
Clothing Accessories	16,9%	1.742
Tracksuits	14,4%	90
Winter Wear	9,5%	2.493
Sleepwear	9,0%	122
Bottomwear	7,5%	3.666
Blazers, Waistcoats and Suits	5,8%	103
Innerwear and Swimwear	5,3%	1.762
Topwear	4,4%	15.310
Kurtas, Ethnic Sets and Bottoms	1,9%	1.180
Men's Footwear	1,1%	920

Ratings are concentrated at the top of the scale (median 3,8, upper quartile 4,1), so a rating filter at 4 would keep a large part of the catalogue. Prices are right-skewed: most products sell for 400-800 and a few exceed 5.000, so price should be used as a range filter rather than a term. The average discount is 50%. Stock status varies noticeably by sub-category: it is more frequently missing for accessories and tracksuits (14-17%) than for footwear (about 1%), so the stock filter will matter more for some categories than others.

4.9 Rankings: brands with well-rated products

The dataset has no sales data, so "top sellers" cannot be measured directly. As a proxy, we count the products with a rating of at least 4 per brand:

Brand	Products rated >= 4
ARBO	623
Pu	567
REEB	493
Marca Disa	323
Gracew	318
PixF	309
Cots	234
Black Beat	220
TOM BU	207
M7 By Metrona	180

This measure reflects catalogue size as much as quality: a brand with many products will appear high even if its average rating is average. A fairer ranking would use the mean rating with a minimum number of products. The truncated brand names (Pu, Black Beat, Marca Disa) also appear here, which confirms the data-quality issue of Section 4.7.

4.10 Named entities

Named-entity recognition was run with spaCy on the first 200 descriptions. The most common entity types were ORG (179), DATE (169), PERSON (86), WORK_OF_ART (39), ORDINAL (39) and PERCENT (25).

The spaCy model is trained mostly on news text, so many product-related strings are labelled incorrectly. For example, brand and product names may be labelled ORG or WORK_OF_ART, and DATE may be triggered by phrases such as "season" or "year". We did not manually verify the labels, so these counts are indicative only. Entity recognition is not used by the search engine in Part 1.

4.11 Validation labels

The validation file has 40 labelled pairs: 13 relevant and 7 non-relevant for query 1, and 10 relevant and 10 non-relevant for query 2. Every pid in the file exists in the catalogue. We checked which product features are common among relevant and non-relevant items:

Query	Label	n	Mention "cotton"	"women" in title
1 (women full sleeve sweatshirt cotton)	1 (relevant)	13	13	13
1	0 (not relevant)	7	7	7
2 (men slim jeans blue)	1 (relevant)	10	8	3
2	0 (not relevant)	10	6	8

Relevant items for query 1 are mostly in the sub-category "Winter Wear" (13 of 13 in the earlier analysis), and relevant items for query 2 are in "Bottomwear" (10 of 10).

Findings for the design of Part 2.

- "cotton" does not discriminate in query 1. All 20 products mention cotton, relevant or not. A term-frequency weight for *cotton* will therefore not change the ordering for this query.
- Gender in the title discriminates in query 2. Only 3 of the 10 relevant products say "women" in the title, against 8 of the 10 non-relevant products. The title field should therefore carry the gender signal, and the query term *men* must reach the title index.
- Titles alone are not sufficient. Two products with the identical title "Full Sleeve Printed Women Sweatshirt" have different labels. The description and product details must be used to tell them apart, which motivates indexing all text fields rather than only the title.
- Labels are small. With 40 pairs, any evaluation metric will have high variance. Part 2 results should be reported with this caveat.

5. Assumptions and limitations

- Sentence splitting relies on NLTK Punkt, which was trained on general text. Product descriptions with missing spaces after full stops may be merged into one sentence.
- Brand truncation and missing values are assumed to come from the scraping process.
- No sales data. "Top sellers" are approximated by well-rated products.
- Evaluation labels are tiny (40 pairs, two queries), so conclusions from them are provisional.

6. Use of AI tools

We used an AI assistant to help structure this report. We also used it to draft some of the code, but each team member participated, ran it, and can explain each function.